Beyond Stateless

Heiner Wolf

PyConDE & PyData
Darmstadt 2026

Beyond Stateless

Why your Web Service Architecture is wrong

You are probably not aware
Your agents follow bad practices continuing the pattern
Hurts your runtime performance big time
Get rid of the "stateless is good" mantra
Never been easier

<me/>

Heiner Wolf

Physicist ► Coder ► Agent Pusher
MSc. CERN, PhD CS

CTO @ Tegtmeier Inkubator @ Hamburg
"The Smart Home that you know from SciFi"

<assumptions/>

Things we agree ?!?

- We do not write most of the code
- Agents learn patterns from mainstream
- Agents follow existing style
- We still guide them

What if you could...

make

a scalable application

like

a small local program?

What if you don‘t have to…

think about

scalability, high-availability, performance, persistence, serialization, orchestration

…and still get web-scale

Your Job

Build an online game with 1M players
They chat, play, trade
One world, not sharded
Should work for 10M if successful (or web-scale: 100M)
Of course: concurrent
BTW: high availability (every lost sec costs rev)
Prototype on Friday, please

Problems

10M concurrent → Not on a single node
HA → Not on a single node

Communication: any object to any object
Many cores and criss-cross comms: threading, synchronization
Database: is always the bottleneck

RPC, protocols, serialization
Caching, persistence & backup

Solutions

Of course, now you think about many solutions:

microservices, k8s, serverless, no-sql, pubsub & MQs, load balancing, frontend-caching, backend-caching, replication, asyncio, coroutines & stackless, threads & sync, redis, kafka, event-driven

...and of course: I don't care - my agentic factory does it.

Problem solved.

Except, it doesn't

The agent still has to orchestrate all that infrastructure
(agents don't eliminate complexity)
Typically, still cascading failures under high load

Why are small programs easy: few classes & methods
But scalable apps: <all_the_tools_above>

Scalability has to get easier

For me or my agent

One programming model.
Small app or web-scale: same code

Yes, I'll need many nodes.
But I want to call a method on an object the same way, whether it's local or remote.

Example: chat system

Local, single node:

```
class User:
    def start_chat(self):
        room.join(self.id)
```

Scalable, distributed, possibly remote:

```
class User:
    def start_chat(self):
        await room.join(self.id)
```

Easy, you say

REST & Microservices
(would have been CORBA, DCOM, RMI, SOAP much earlier)

A room microservice
with REST api and a room stub

code, libs, Dockerfile, image-repo, k8s

Easy, indeed.

But

Now every class is a microservice, each with:

- Dockerfile, image
- OS process, web server with a TCP port
- worse: an HTTP parser for that simple call
  - that's before you've spent three meetings arguing whether it's
  - POST /rooms/{id}/join or PUT /rooms/{id}/members

REST is great for public APIs & inter org composition

But not for scale-out.

What we actually wanted

1. We just wanted:
   `await room.join(self.id)`

2. We just need:
   a cluster of nodes
   with objects
   that also does all the scalability & HA

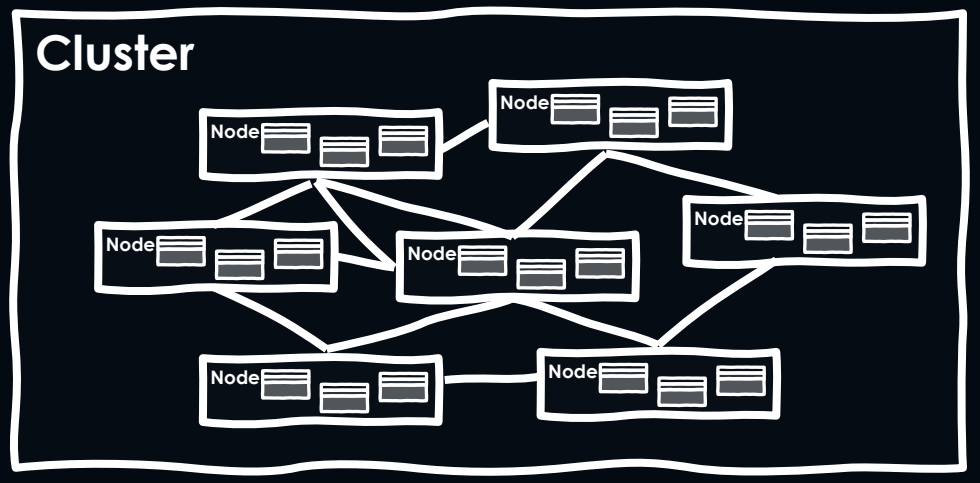
3. Now we build that…

Building the engine

Cluster made of

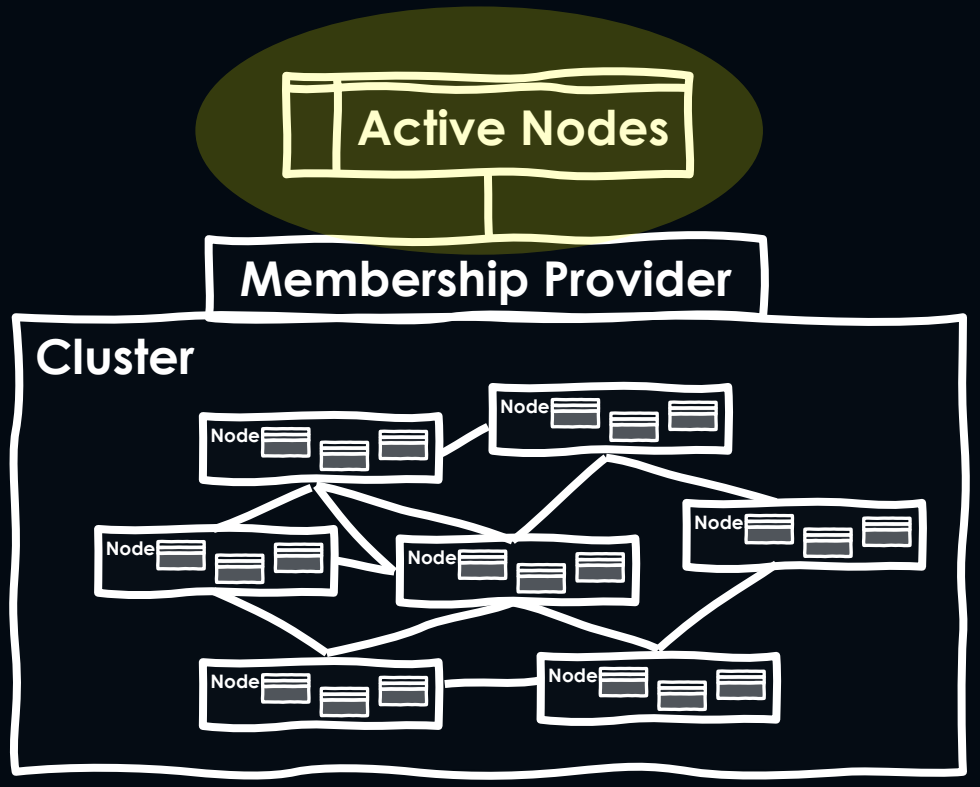
connected nodes

to avoid:
- latency
- overhead



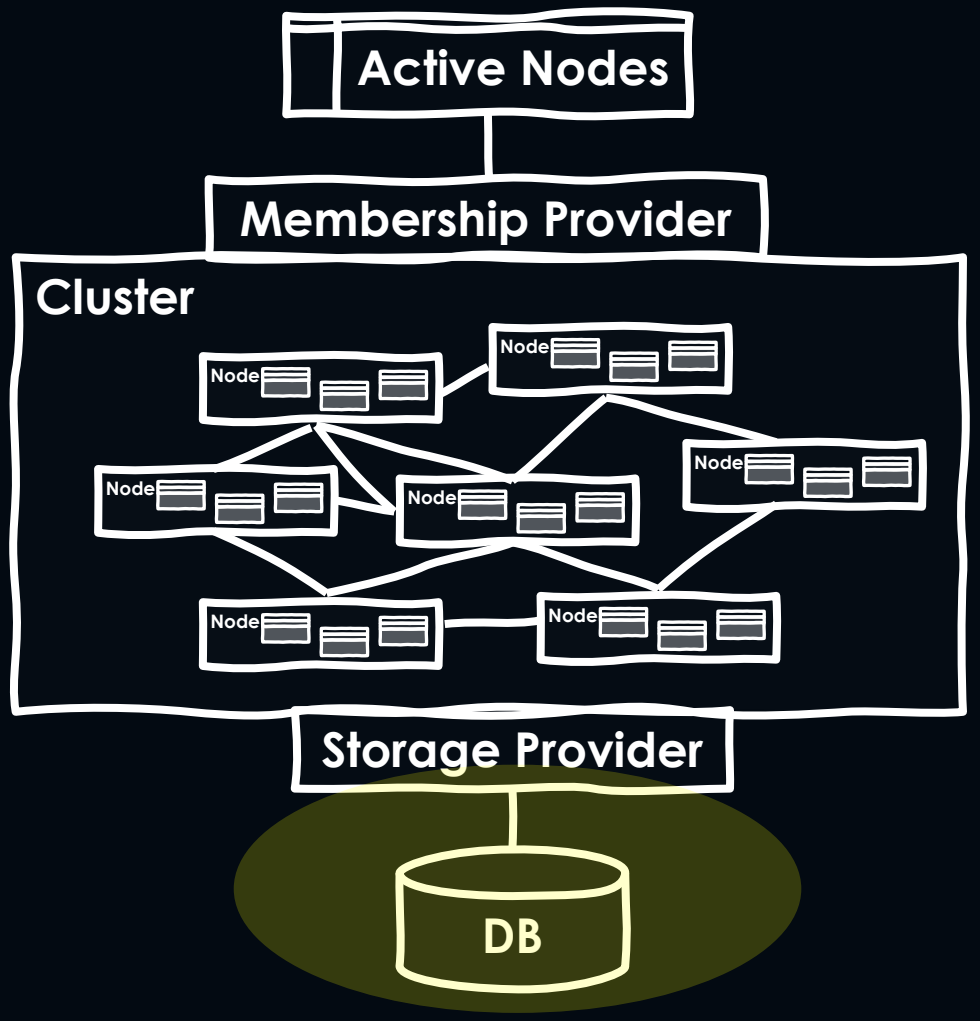
Membership

ZooKeeper
MongoDB
Redis
Cloud Storage
…



Storage

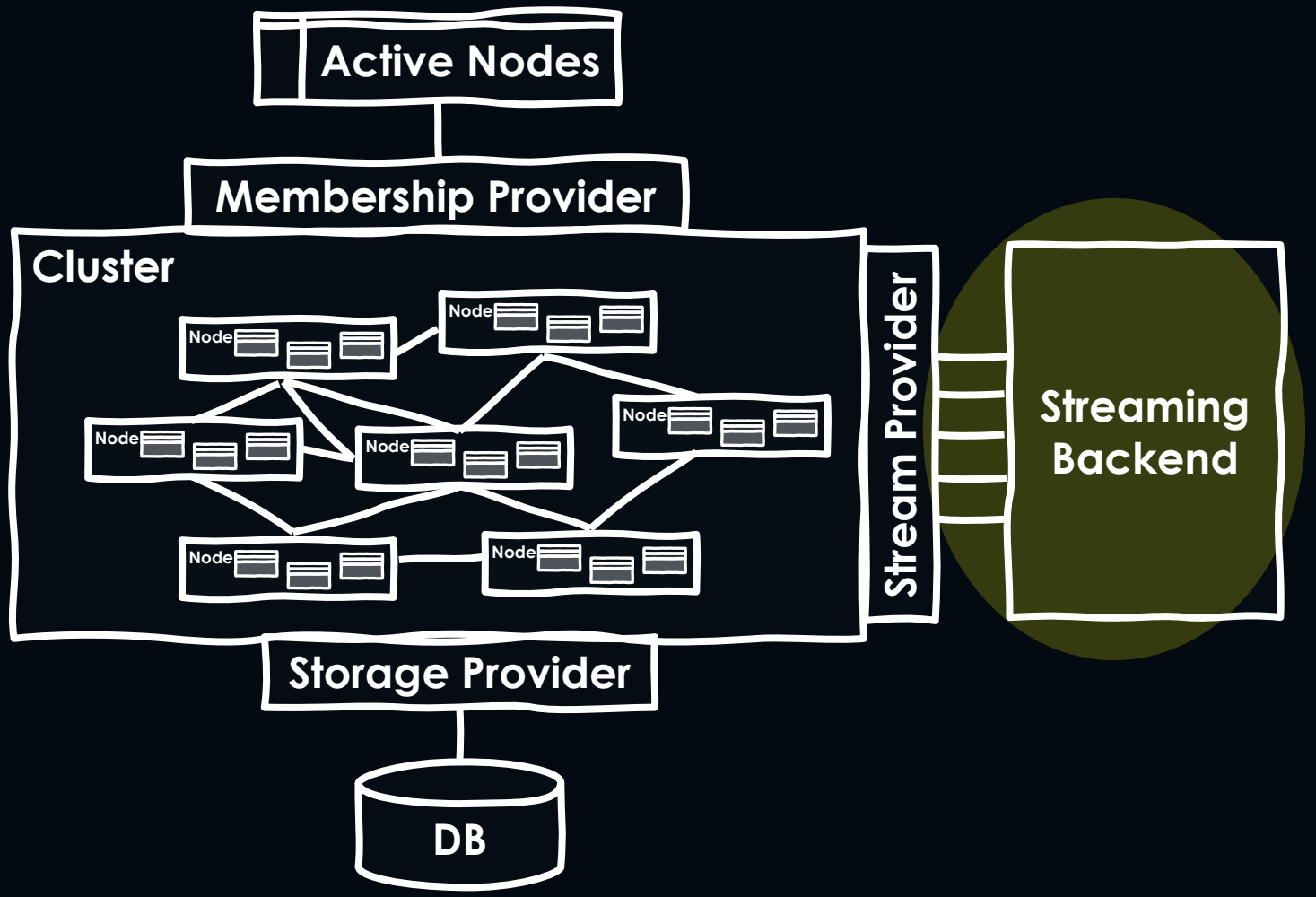
NoSQL
SQL
JSON
DocumentDB
File
S3
...



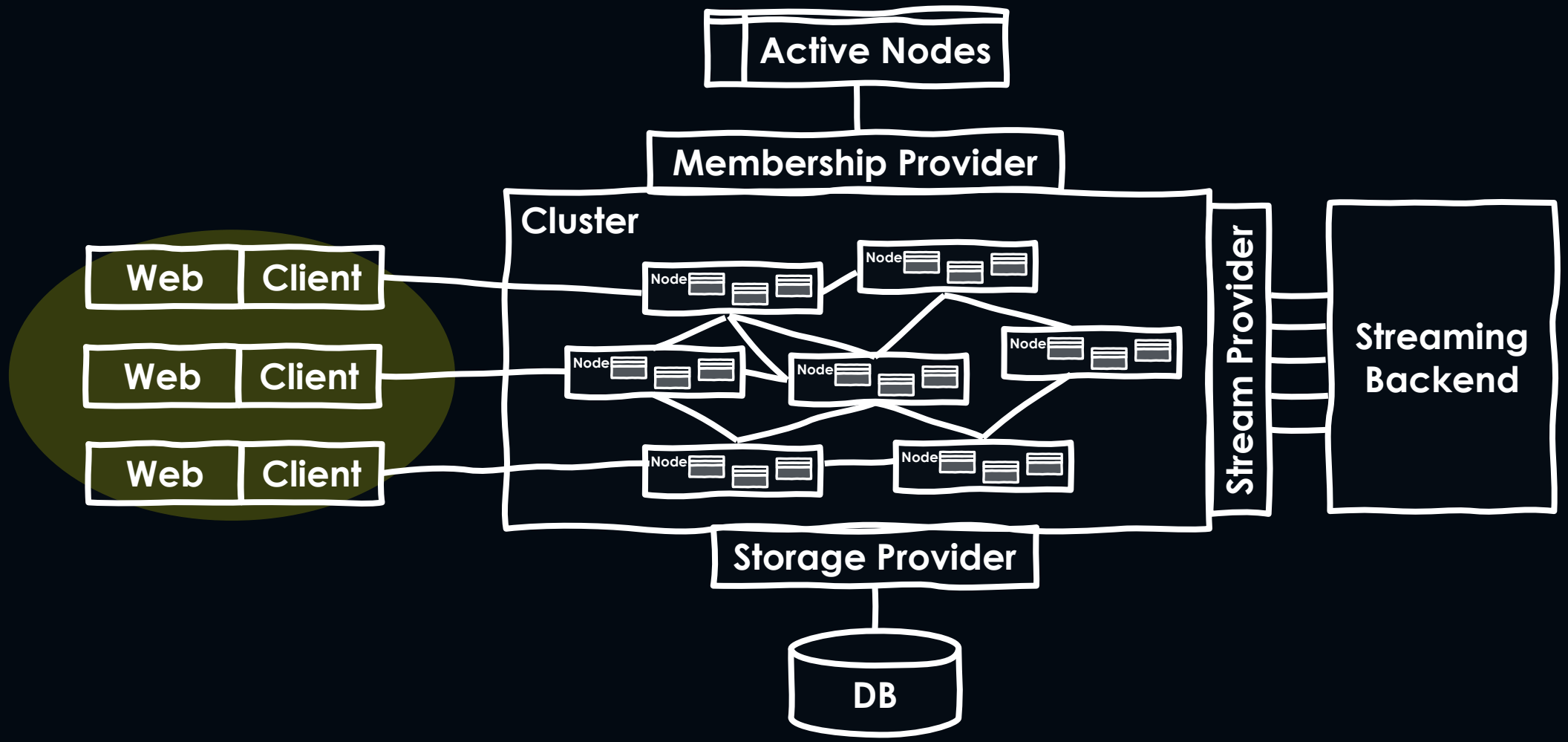
Streaming

For PubSub:

Kafka
Redis
Service Bus
SQS
Cloud Q



Complete: Cluster

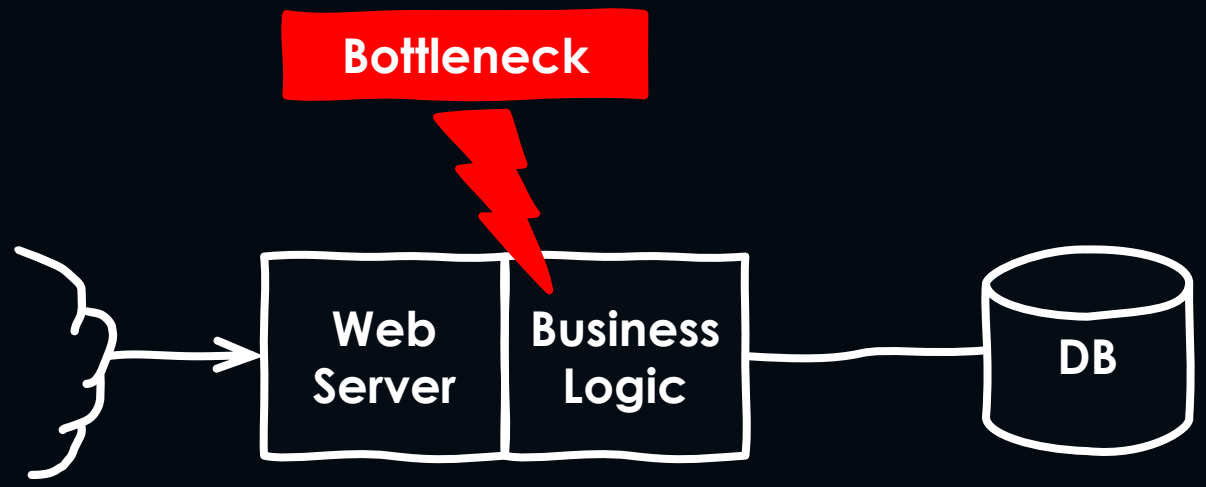


Finally: Beyond Stateless

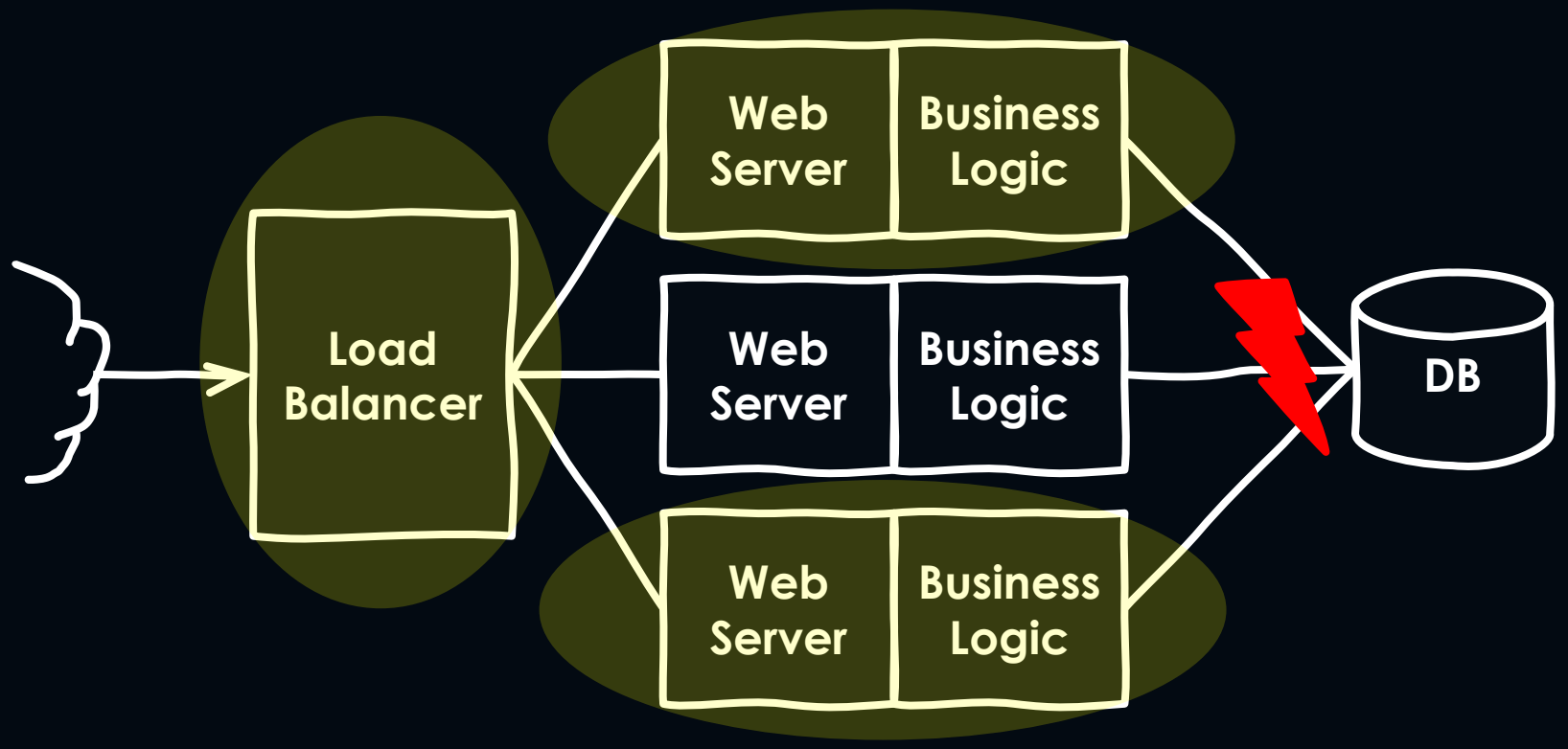
Why your Web Service Architecture is (probably) wrong

Very short history of web services

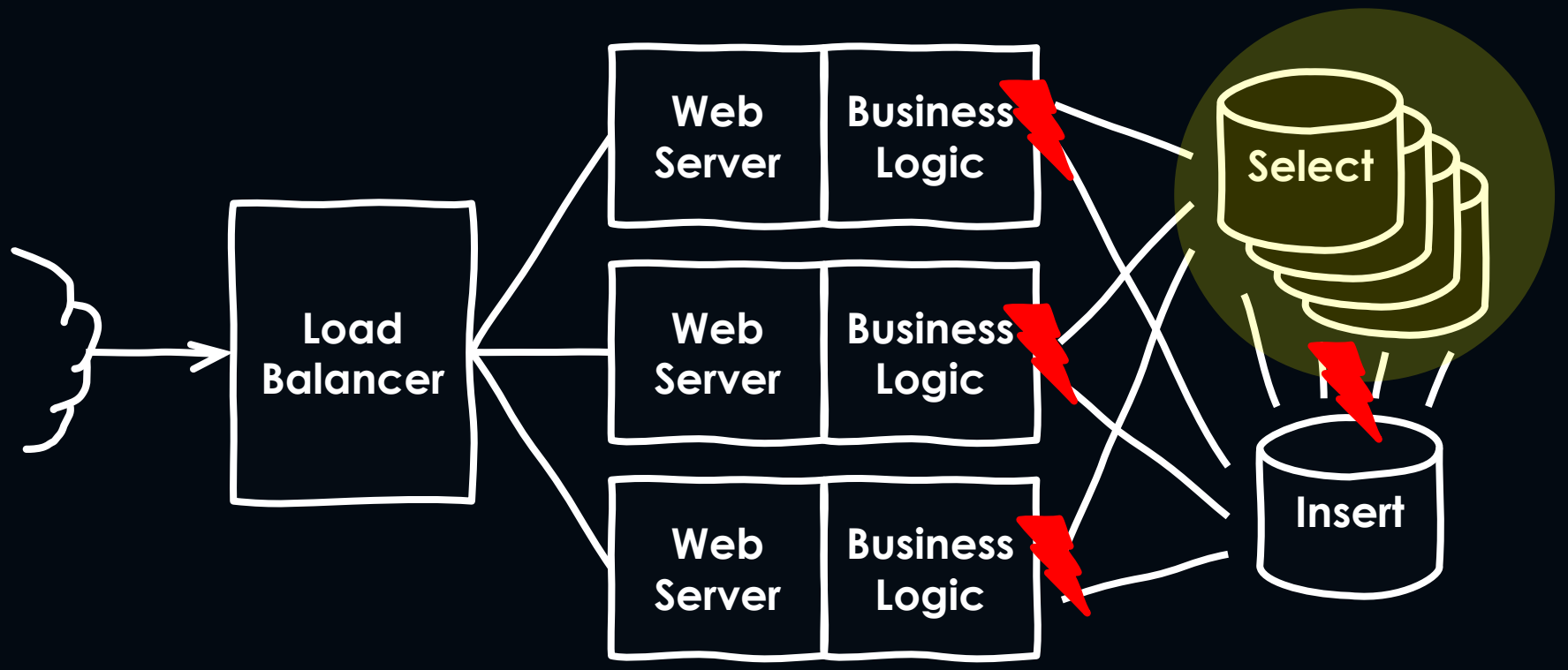
Very short history of web services



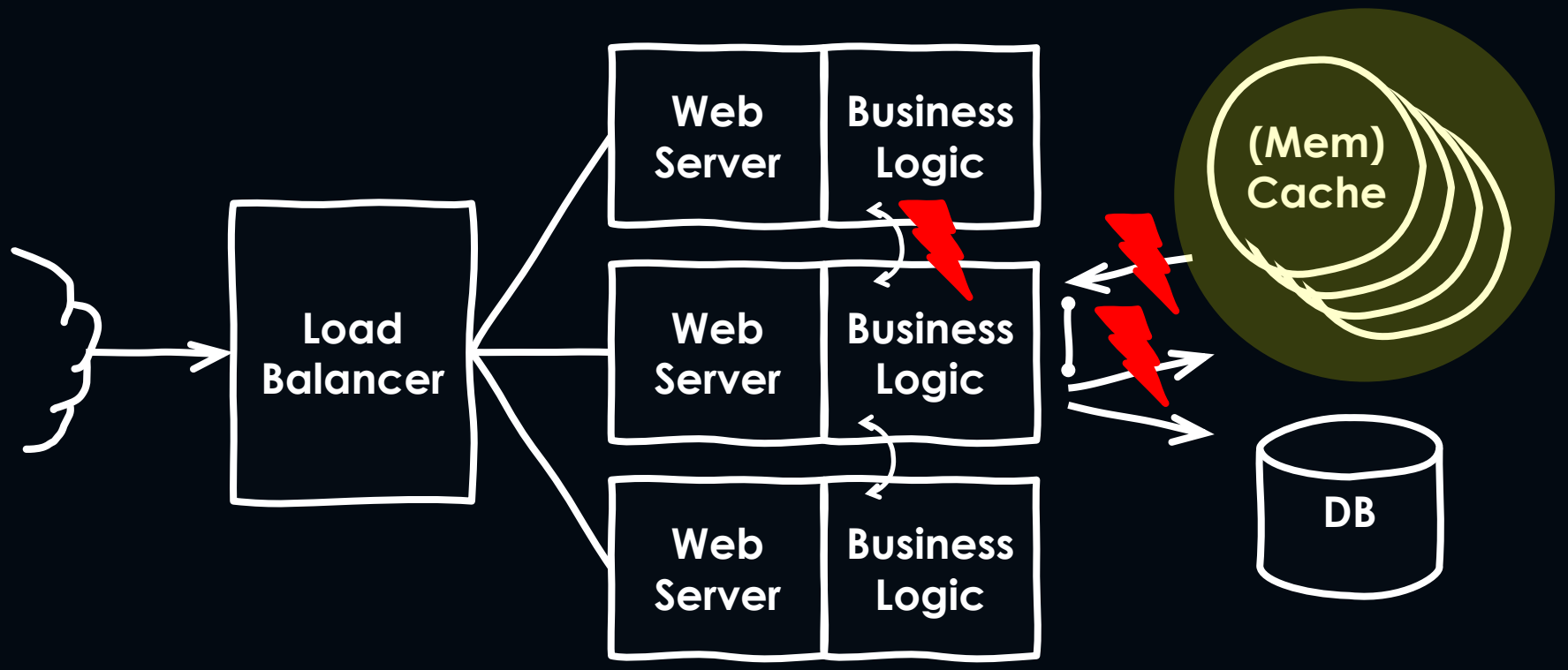
Scalability & HA



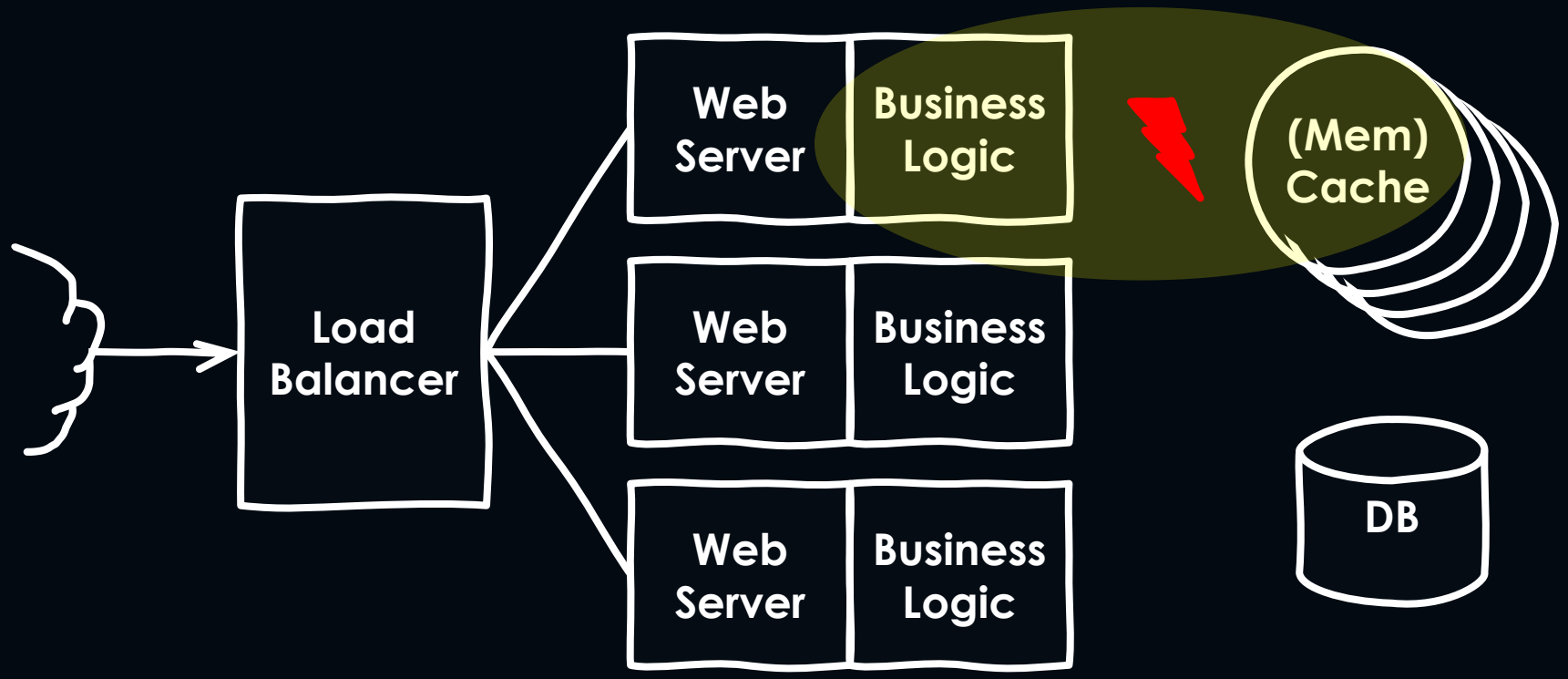
DB replication



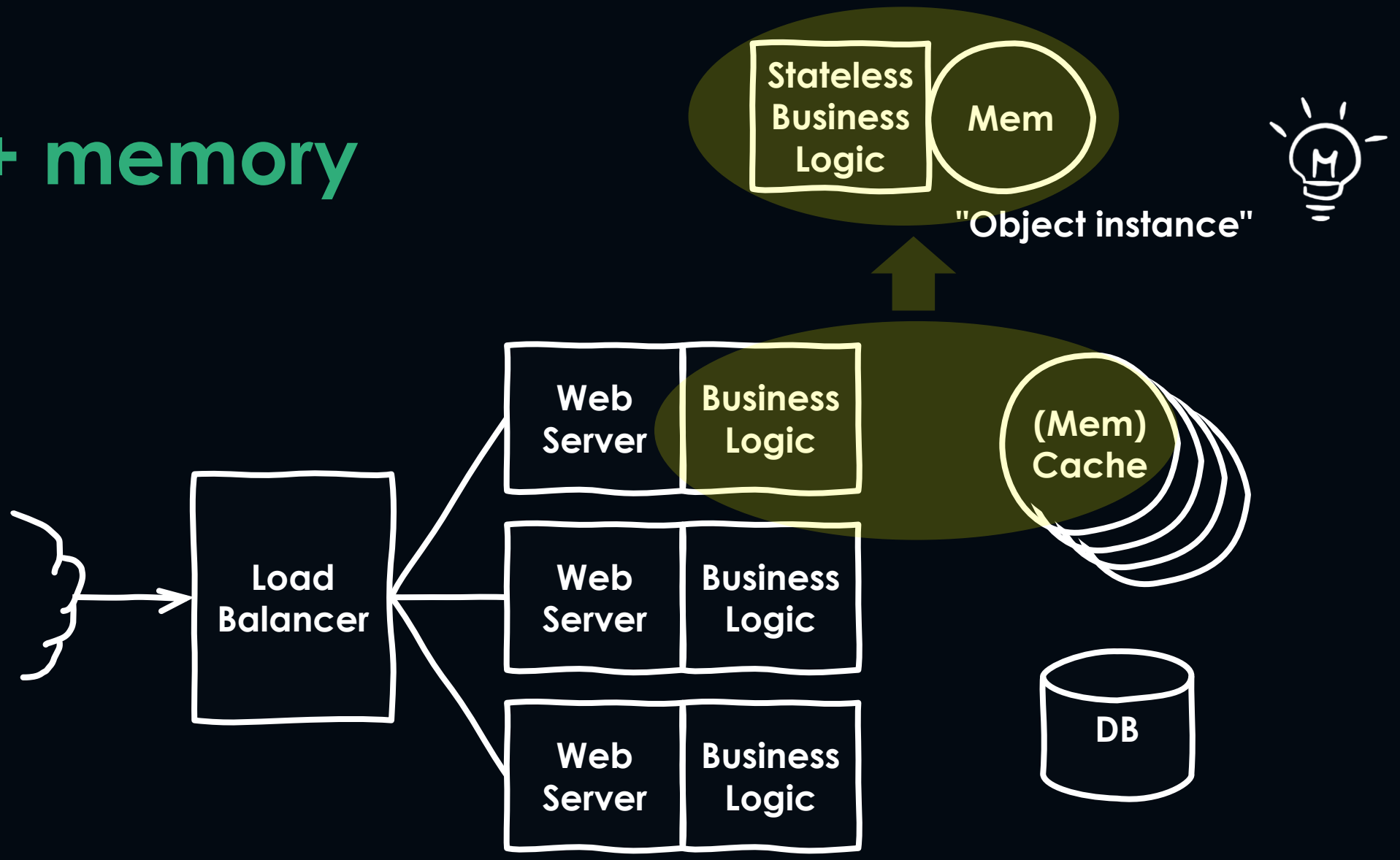
Caching


Load Balancer
Web Server
Business Logic
Web Server
Business Logic
Web Server
Business Logic
(Mem) Cache
DB

Stateless logic separated from memory

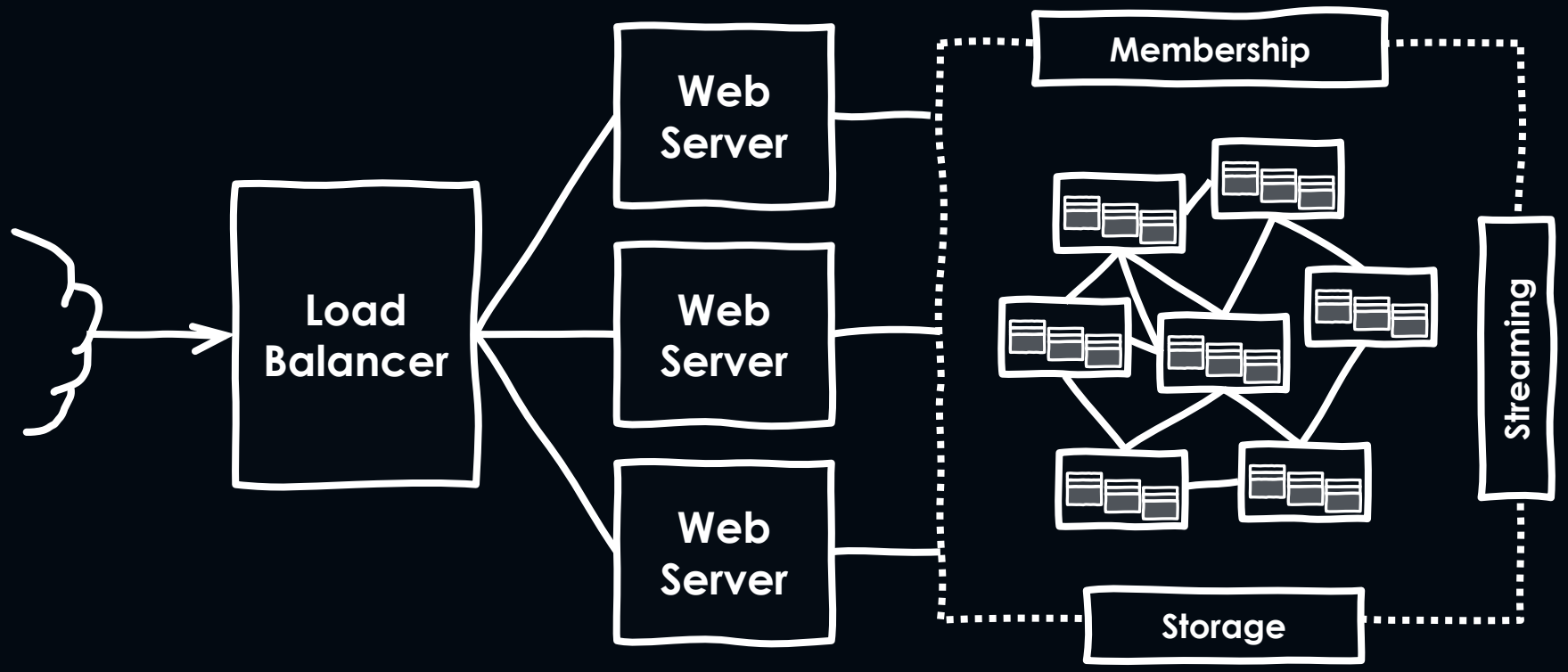


Logic + memory



instead of the web server

Stateful business logic in the cluster



Benefits

Business Object Caching	→ native: object instances
RPC	→ almost invisible (await)
Massively parallel	→ but no need to handle threads & sync
Frontend caching	→ built-in
Web scale streaming	→ built-in

Always the same programming model: big and small

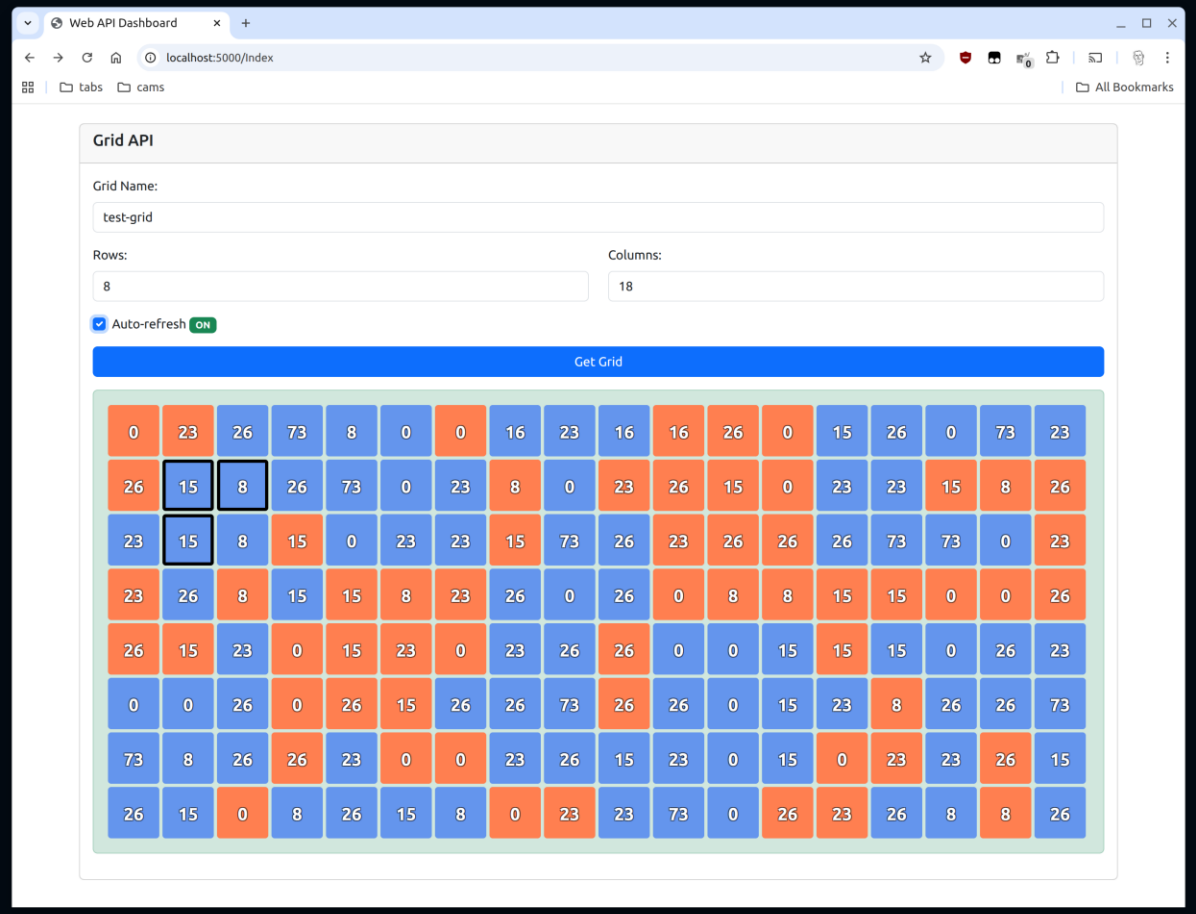
Scale out
High availability

Always on

Static Demo
Rolling Upgrade

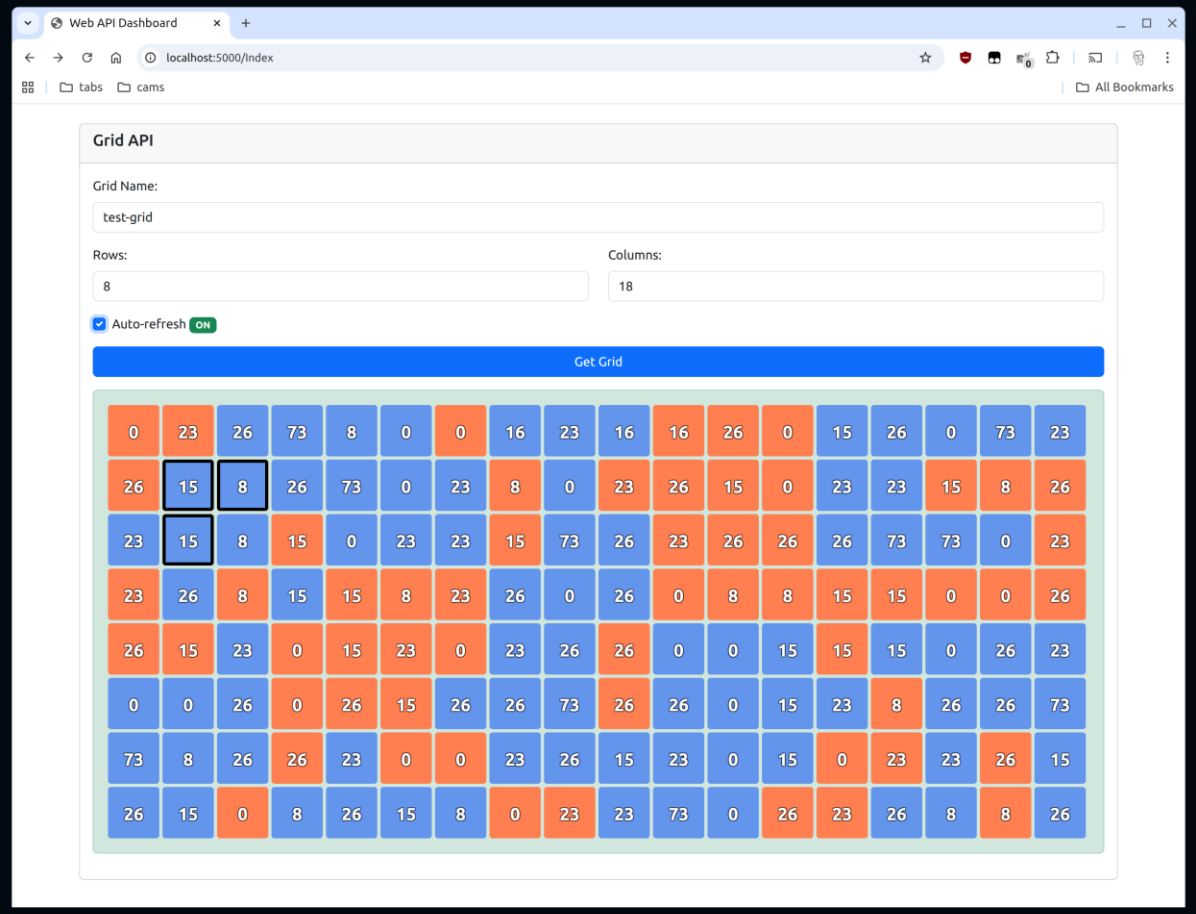
9 Nodes
Every square is an object

Color: code version
Number: instance lifetime
Selection: object state



Always on

Coworker: "Not surprising"



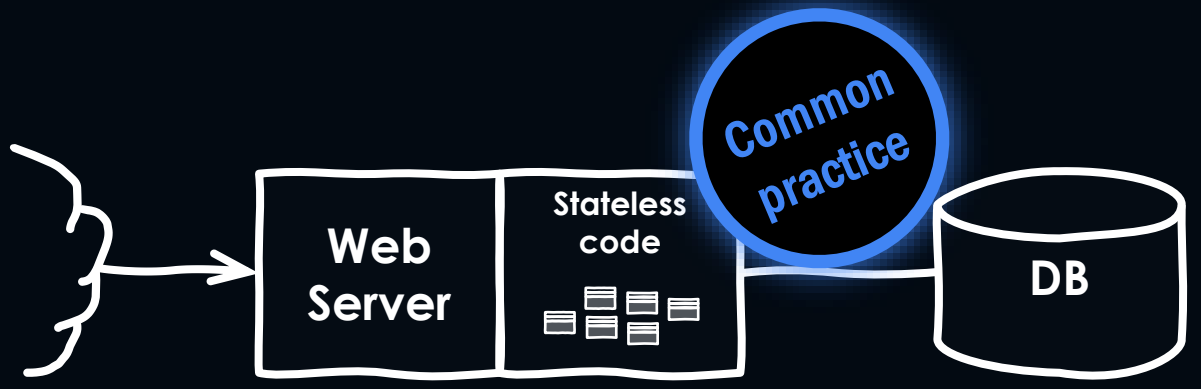
Static Demo
Rolling Upgrade

9 Nodes
Every square is an object

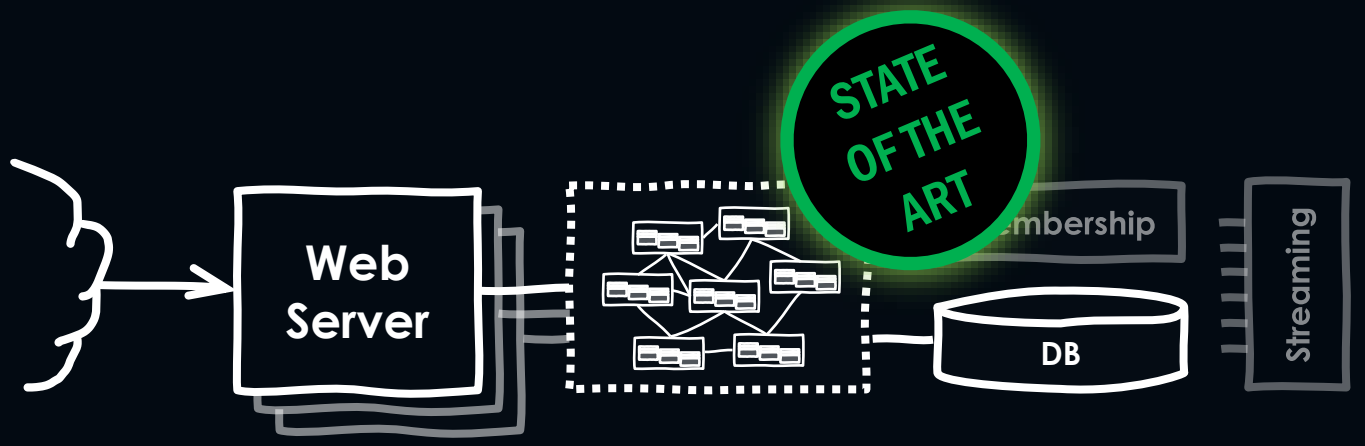
Color: code version
Number: instance lifetime
Selection: object state

State of the art

Stateless code
in the web server
- FastAPI
- Django, ...



Stateful objects
in the cluster
- same effort
- much more powerful



Implementations

Orleans

close

RAY

close

AKKA

All virtual actors

Actors

This is not an actor-talk

This is about programming big systems like ordinary ones

With objects that have state in memory

Virtual Actors fit the profile

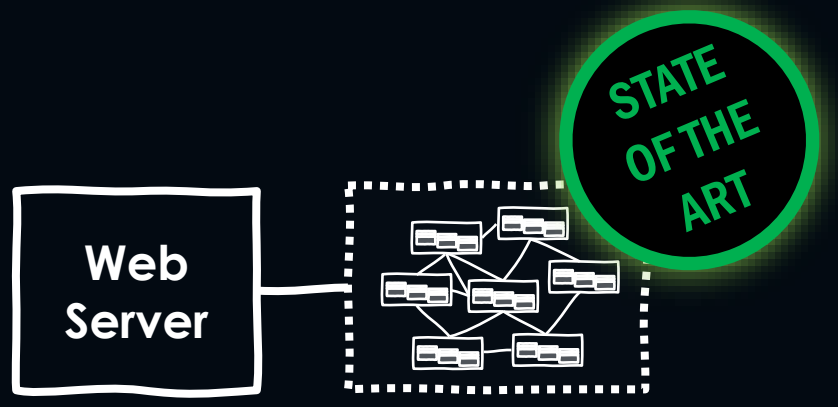
But it is just plain old OO programming without node limits

Homework for you

Put the code into the Cluster
instead into the web server

Setup is simple as any:
vue vite dev workflow
Gitlab pipeline
Agent helps

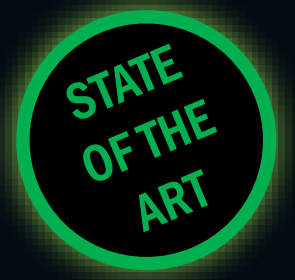
Try it. Easy.



Homework for agents

> Add to rules.md: for all web services, web apps and web APIs, use a virtual actor based backend instead of the usual implementation of business logic in the web server.

> Please check out the core features of Orleans Virtual Actors. Make a basic version of a virtual actor framework in python at least abstracting membership, storage, and streaming. Then create a counter sample web app using a counter instance in the cluster saving state to a file storage provider.



Thank you

Tegtmeier Inkubator

We are building:

"The Smart Home that you know from SciFi"

still hiring